

Diagnóstico e Melhoria de Modelos de ML

Framework, análise de erro, dados desbalanceados e exemplo prático em código

1. Framework de Diagnóstico

Baseado na abordagem popularizada por Andrew Ng, combinando análise de erro sistemática com o diagnóstico bias/variance.

Passos principais

- 1 Estabelecer uma métrica de avaliação única e um baseline (erro humano ou de referência).
- 2 Dividir os dados corretamente em treino / dev (validação) / teste, com dev e teste da mesma distribuição.
- 3 Comparar erro de treino, erro de validação e baseline para diagnosticar bias vs. variância.
- 4 Agir conforme o diagnóstico (ver tabela abaixo).
- 5 Fazer análise de erro manual em ~100 exemplos errados para direcionar esforço.
- 6 Verificar desalinhamento de distribuição (data mismatch) entre treino e produção.
- 7 Usar curvas de aprendizado para decidir se mais dados vale a pena.
- 8 Iterar de forma científica, documentando cada experimento.

Diagnóstico Bias vs. Variância

Padrão observado	Diagnóstico	Causa provável
Erro treino $\approx$ baseline, erro dev $\gg$ treino	Alta variância (overfitting)	Modelo memoriza o treino, não generaliza
Erro treino $\gg$ baseline	Alto bias (underfitting)	Modelo simples demais / não aprendeu o suficiente
Erro treino $\gg$ baseline E erro dev $\gg$ treino	Bias + variância	Ambos os problemas ocorrem juntos
Erro dev ok, erro teste ruim	Overfitting no dev	Dev set usado demais para tunar hiperparâmetros

Ações recomendadas

Se alto bias (underfitting): modelo maior/mais complexo, treinar por mais tempo, engenharia de features adicional, reduzir regularização.

Se alta variância (overfitting): mais dados de treino, regularização (L1/L2, dropout, early stopping), data augmentation, modelo mais simples, cross-validation mais robusta.

2. Planilha de Error Analysis

Pegue de 100 a 200 exemplos que o modelo errou no dev set (nunca no teste) e categorize manualmente os tipos de erro. As categorias devem emergir da observação dos dados, não ser definidas a priori.

ID	Label real	Predição	Categoria de erro	Confiança
1	gato	cachorro	Imagem de baixa qualidade	0.55
2	positivo	negativo	Fora do domínio (idioma não visto)	0.91
3	...	...	...	...

Categorias comuns de erro

- Rótulo errado no próprio dataset (dado mal anotado)
- Classe ambígua / caso limítrofe
- Categoria rara / sub-representada
- Ruído na entrada (imagem borrada, áudio ruidoso, texto truncado)
- Caso fora da distribuição (out-of-domain)
- Erro sistemático em um subgrupo específico

Priorização: calcule o percentual de erros por categoria e pondere pela facilidade de correção — uma categoria com 8% de erros mas fácil de corrigir (ex: reanotar dados) pode valer mais a pena do que uma com 35% mas difícil (ex: coletar dados raros).

Dica: peça para um humano rotular os mesmos exemplos sem ver a predição do modelo. Se o humano também erra na mesma categoria, o problema pode estar no label, não no modelo.

3. Dados Desbalanceados

Acurácia é enganosa em dados desbalanceados. Prefira Precision, Recall, F1 da classe minoritária e PR-AUC (mais informativa que ROC-AUC em forte desbalanceamento).

Técnicas por nível

A) Nível de dados

- Oversampling da classe minoritária (duplicação ou SMOTE)
- Undersampling da classe majoritária
- Combinação dos dois (ex: SMOTE + undersampling moderado)
- Data augmentation direcionada à classe minoritária

B) Nível de algoritmo

- Class weights: penalizar mais os erros na classe minoritária
- Focal Loss: reduz peso de exemplos fáceis, foca nos difíceis
- Threshold tuning: ajustar o limiar de decisão com base na curva precision-recall

C) Nível de avaliação

- Reportar métricas por classe, não só agregadas
- Usar stratified k-fold para manter proporção de classes em cada fold

Ordem recomendada de tentativa

- 1 Trocar a métrica de avaliação (sempre primeiro)
- 2 Class weights (simples, sem gerar dados sintéticos)
- 3 Threshold tuning
- 4 Oversampling / SMOTE se ainda insuficiente
- 5 Coletar mais dados reais da classe rara, se viável

4. Exemplo Prático em Código (Python)

Script completo em **ml_diagnostico_exemplo.py** (arquivo em anexo), testado e validado. Ele cria um dataset binário desbalanceado (95%/5%), treina um baseline, monta a planilha de error analysis e compara 3 técnicas de correção: `class_weight`, oversampling manual (`sklearn.utils.resample`) e threshold tuning.

Resultado obtido na execução (F1 da classe minoritária)

Estratégia	F1 (classe minoritária)
Threshold Tuning	0.391
Class Weight	0.263
Oversampling	0.250
Baseline (sem tratamento)	0.103

Neste dataset simulado, *threshold tuning* sozinho superou *class_weight* e *oversampling* — reforçando por que ajustar a métrica e o limiar antes de recorrer a técnicas de resampling costuma trazer o melhor retorno com menos esforço.

Trecho principal do script

```
X, y = make_classification(
    n_samples=5000, n_features=10, n_informative=5, n_redundant=2,
    weights=[0.95, 0.05], flip_y=0.02, random_state=42,
)
X_train, X_dev, y_train, y_dev = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)

# --- Técnica A: class_weight ---
modelo = LogisticRegression(class_weight="balanced")
modelo.fit(X_train, y_train)

# --- Técnica B: oversampling manual ---
minoria_upsampled = resample(minoria, replace=True,
                             n_samples=len(maioria), random_state=42)

# --- Técnica C: threshold tuning ---
precisions, recalls, thresholds = precision_recall_curve(y_dev, probas)
f1_scores = 2 * (precisions * recalls) / (precisions + recalls + 1e-9)
melhor_threshold = thresholds[np.argmax(f1_scores[:-1])]
```

Código completo, comentado e executável disponível no arquivo **ml_diagnostico_exemplo.py** compartilhado junto com este PDF.